

Estimating Performance

Bal Krishna Nyaupane

Assistant Professor

Department of Electronics and Computer Engineering

Paschimanchal Campus, IOE, TU

bkn@wrc.edu.np

Underfitting

- A machine learning algorithm is said to have underfitting when it *cannot capture the underlying trend of the data*.
- Its occurrence simply means that our model or the algorithm *does not fit the data well enough*.
- It usually happens when we *have less data to build an accurate model*.
- A learner that underfits the training data *will miss important aspects of the data*, and this will *negatively impact its performance* in making accurate predictions on new data it has not seen during training.
- ***Underfitting: Larger error on training and test sets.***

Underfitting

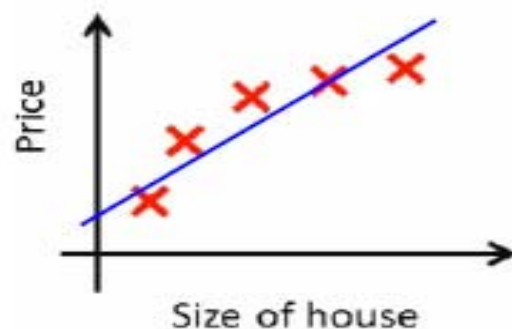
■ *For examples:*

- The learning time may be prohibitively large, and the learning stage was *prematurely terminated*.
- The learner *did not use a sufficient number of iterations*.
- The learner *tries to fit a straight line to a training set* whose examples *exhibit a quadratic nature*.

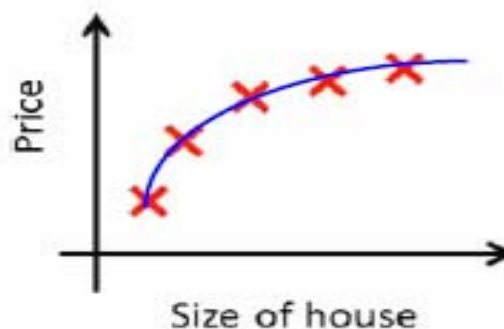
Overfitting

- A statistical model is said to be overfitted, when we *train it with a lot of data* (just like fitting ourselves in an oversized pants!).
- When a model gets trained *with so much of data*, it starts learning from the *noise and inaccurate data entries in our data set*. Then the model does not categorize the data correctly, because of too much of details and noise.
- If you use a *higher order polynomial function* (non-linear function) for your fitting curve than that *exhibited by the training set*, then there may *exist possibly several different subsets of polynomials* that can be used to adequately learn the training set.
- The learner *has a degree of freedom to choose any subset of polynomials* from the set to learn the training set.
- ***Overfitting: Small error on training set, but large error on unseen examples.***

Regression Example



- Let x =Size of house.
- Linear fit:
 $\theta_0 + \theta_1 x$
- A line has a poor fit of the training data and the training error is relatively large.
- Similarly, the machine would perform poorly on new inputs.
- Underfitting/Low capacity

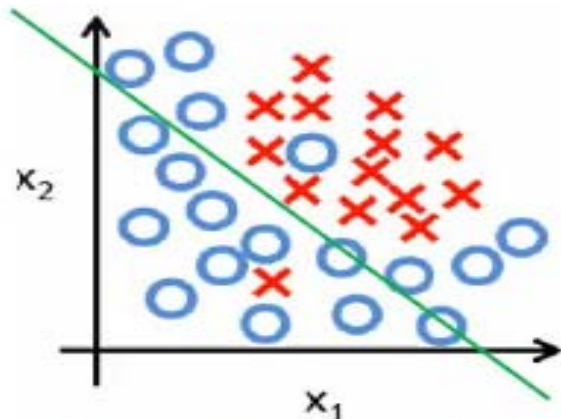


- Let x =Size of house.
- Quadratic fit:
 $\theta_0 + \theta_1 x + \theta_2 x^2$
- Parameters may not fit the data exactly, but the machine would perform relatively well on new data (assumes the training set is representative of the new inputs).
- Optimal fit/Optimal capacity



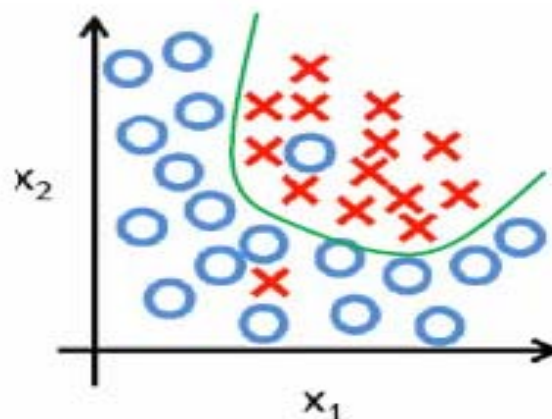
- Let x =Size of house.
- Polynomial fit:
 $\theta_0 + \theta_1 x + \theta_2 x^2, \dots, \theta_n x^n$
- Parameters fit the data exactly:
$$J(\theta) = \frac{1}{2m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^i)^2 = 0$$
- But the machine would perform poorly on new data.
- Overfitting/High capacity

Classification Example



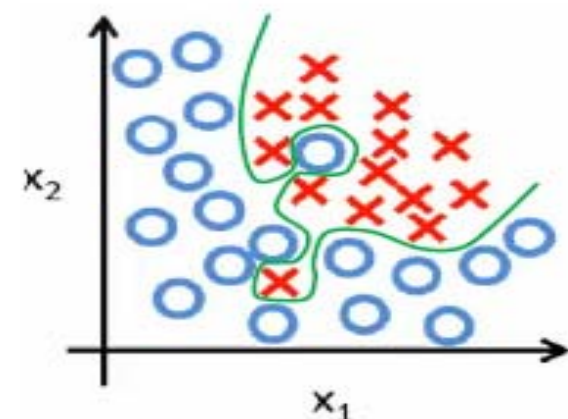
- Let x_1 =Size, and x_2 =Rooms.
- Linear fit:

$$\theta_0 + \theta_1 x_1 + \theta_2 x_2$$
- A line has a poor fit of the training data and the training error is relatively large.
- Similarly, the machine would perform poorly on new inputs.
- Underfitting



- Quadratic fit:

$$\theta_0 + \theta_1 x + \theta_2 x^2$$
- Parameters may not fit the data exactly, but the machine would perform relatively well on new data (assumes the training set is representative of the new inputs).
- Optimal fit.



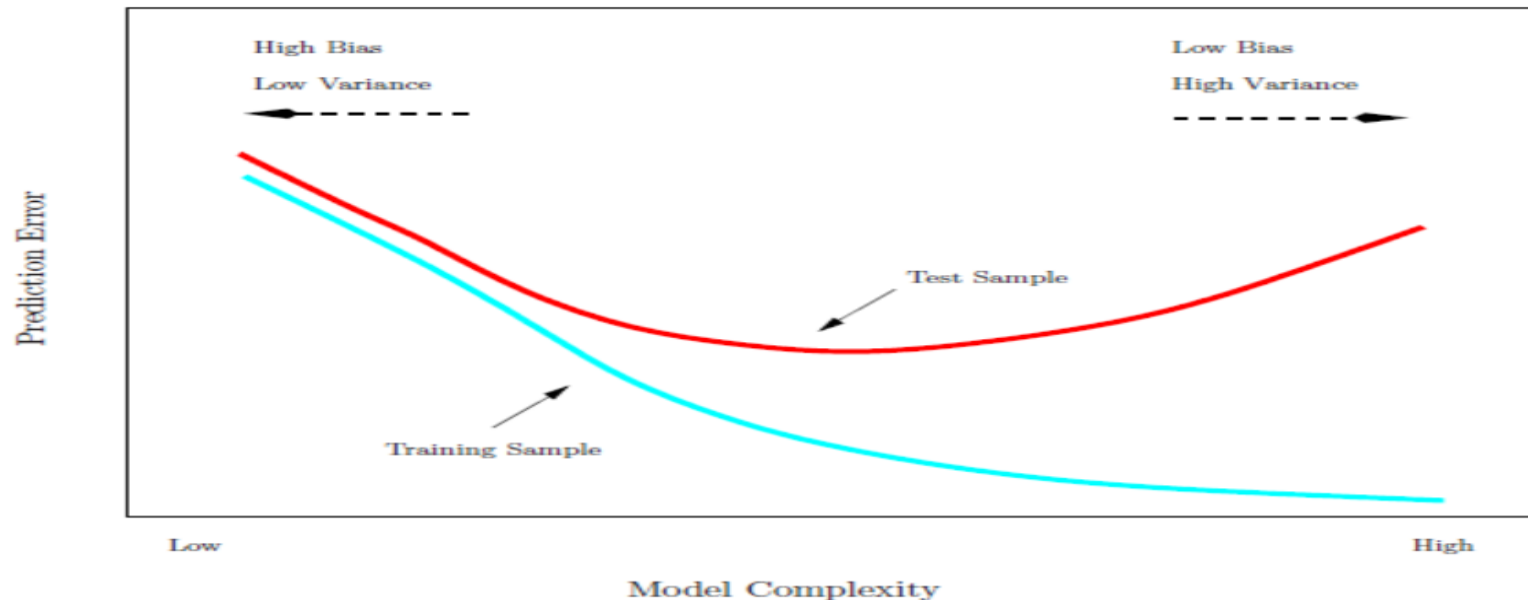
- Polynomial fit:

$$\theta_0 + \theta_1 x + \theta_2 x^2 + \theta_3 x^2 x^3, \dots$$
- Parameters fit the data exactly:

$$J(\theta) = \frac{1}{2m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^i)^2 = 0$$
- But the machine would perform poorly on new data.
- Overfitting.

Training Error versus Test error

- The *test error* is the average error that results from using a statistical learning method to predict the response on a new observation, one that was not used in training the method.
- In contrast, the *training error* can be easily calculated by applying the statistical learning method to the observations used in its training.



What is Generalization?

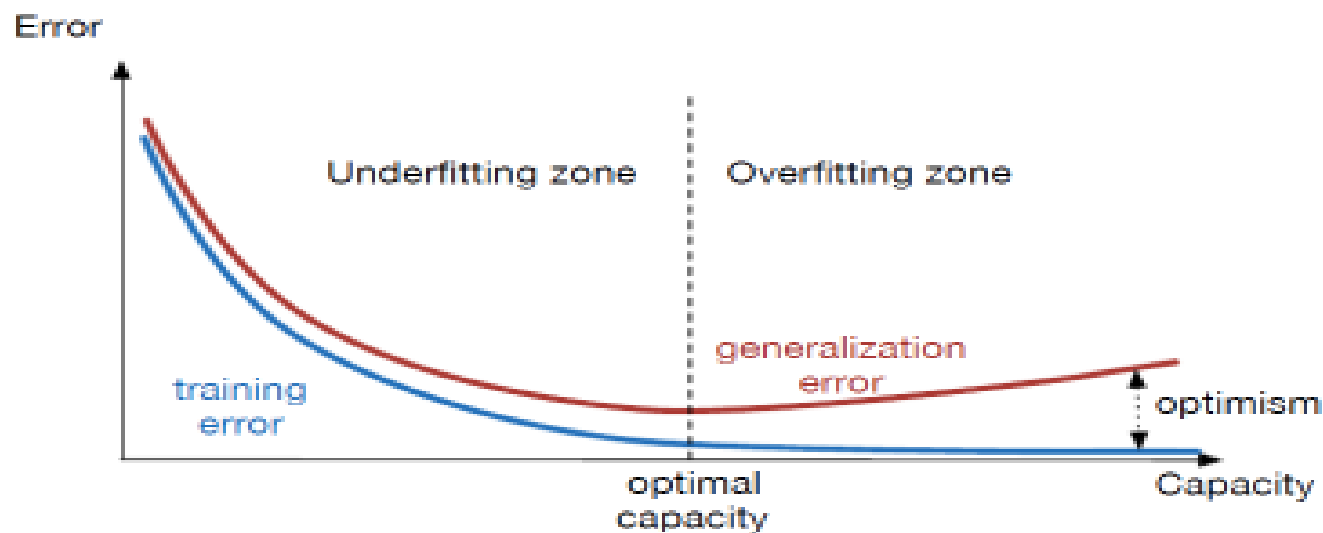
- Generalization is a term used *to describe a model's ability to react to new data*. **Generalization** is a task a learning machine performs when it *makes predictions about new inputs that it has not seen before*.
- A learning machine develops a *generalization ability* by learning *to predict from training examples*. *Generalization performance* is a measure of how well a learning machine performs on new inputs.
- A model's ability to generalize is central to the success of a model. *If a model has been trained too well on training data*, it will be *unable to generalize*. It will make inaccurate predictions when given new data, making the model useless even though it is able to make accurate predictions for the training data. *This is called overfitting*.
- The inverse is also true. Underfitting happens when a model has not been trained enough on the data.

What is Capacity?

- Capacity is the amount of resources a machine learning algorithm *has to model data*. The higher is the *learner's capacity*, the *higher is the percentage of the training examples* the learner *will fit almost perfectly*.
- There are many *ways to change the capacity* of a learner:
 - Change the *number of units in hidden layers* of an artificial neural network
 - Change the number of *training iterations in regression*
 - Change the *degree of non-linearity* of the hypothesis or features.
- There is a *trade-off between capacity and generalization*.
 - A ML algorithm may *have high capacity*, but *poor generalization* ability.
 - *Lowering capacity* might improve a ML algorithm's *generalization ability*.

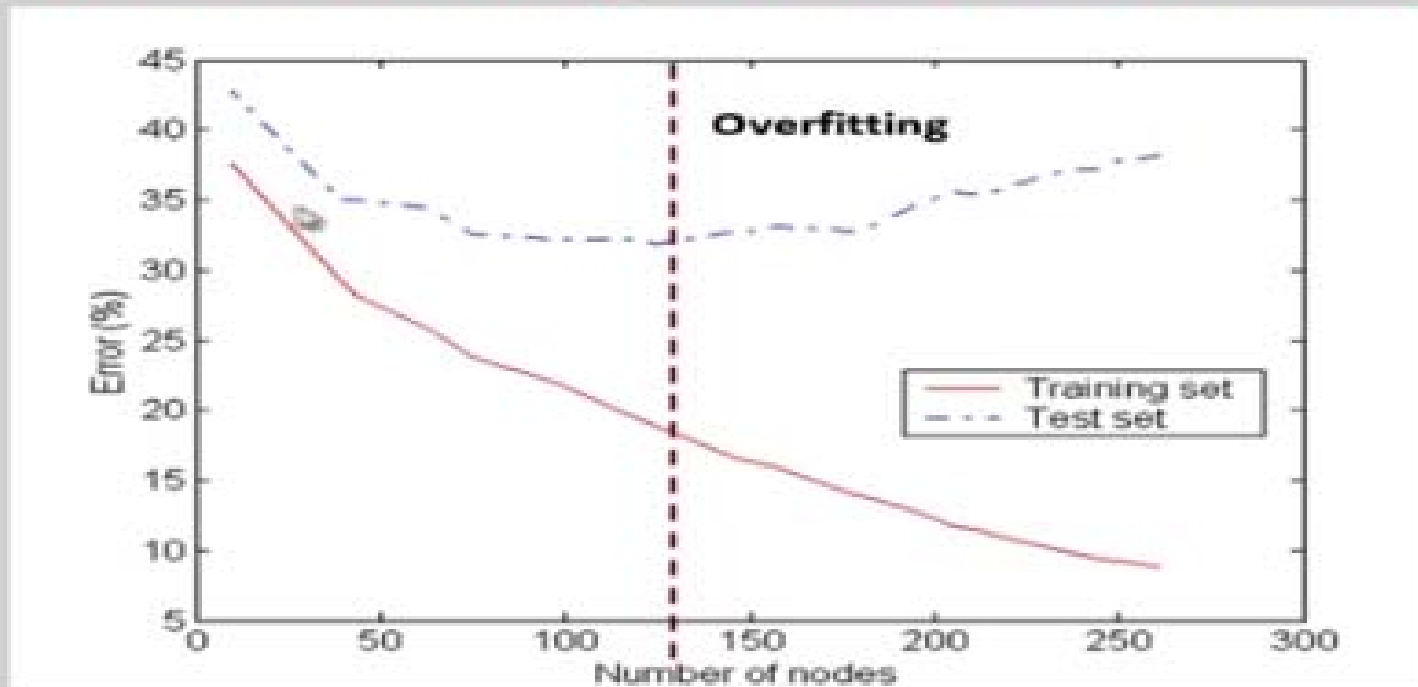
Signs of Underfitting and Overfitting

- As we *increase capacity*, *training error can be reduced*, but the difference between *training and generalization error increases*.
- At some point, the *increase in capacity* causes an *increase in generalization error*, and we *enter the overfitting zone*, where *capacity is too large, above the optimal capacity*.



Signs of Underfitting and Overfitting

Underfitting and Overfitting



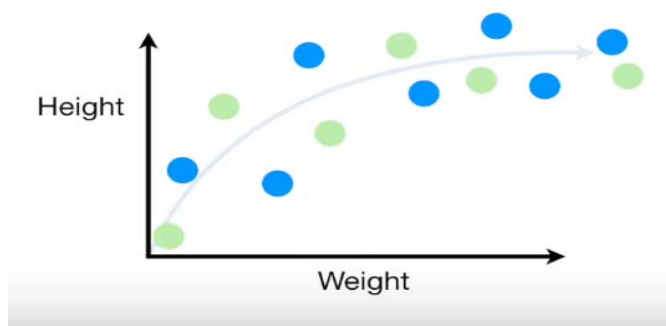
Underfitting: when model is too simple, both training and test errors are large

Bias and Variance

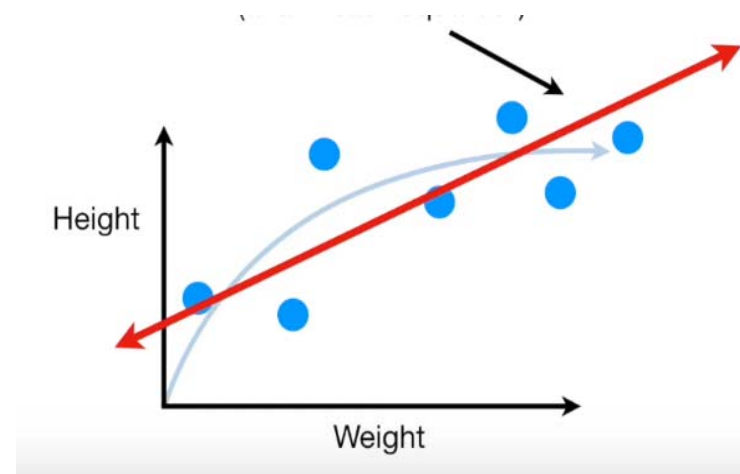
- Bias is the algorithm's tendency to consistently *learn the wrong thing by not taking into account all the information* in the data (underfitting).
- Thus, *parametric algorithms are prone to high bias*. A parametric algorithm is defined as, “A learning model that summarizes data with a set of parameters of fixed size (independent of the number of training examples) is called a parametric model. No matter how much data you throw at a parametric model, it won't change its mind about how many parameters it needs.”
- A linear regression is an example of a parametric algorithm. These are easy to understand but not flexible to learn the underlying signal of the data. Thus, they are inaccurate for complex datasets.
- Examples of high-bias algorithms include *Linear Regression, Linear Discriminant Analysis, and Logistic Regression*.

Bias and Variance

- Wikipedia states, “... variance is an error from sensitivity to small fluctuations in the training set. ***High variance can cause an algorithm to model the random noise in the training data***, rather than the intended outputs (overfitting).”
- Variance is the algorithm’s tendency to learn random things irrespective of the real signal ***by fitting highly flexible models that follow the error/noise in the data too closely*** (overfitting).

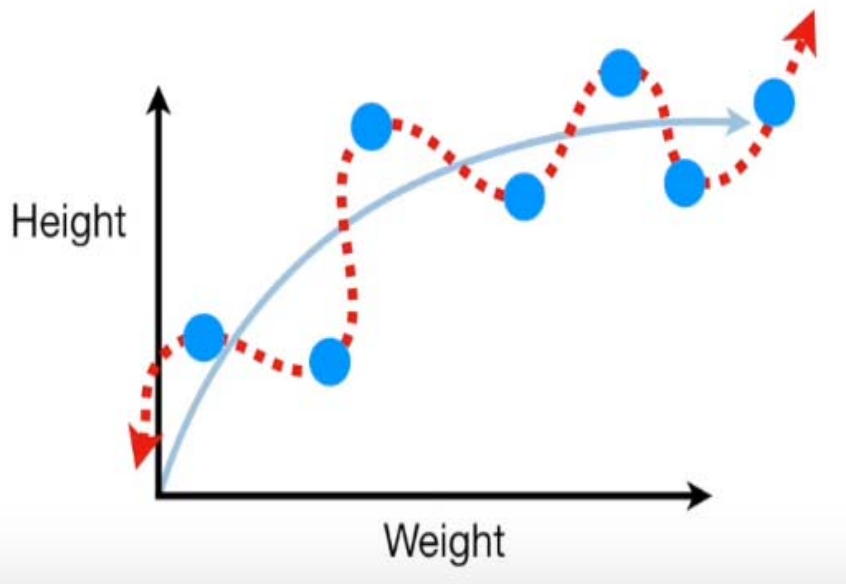


Blue dots: Training set
Green dots: Testing set

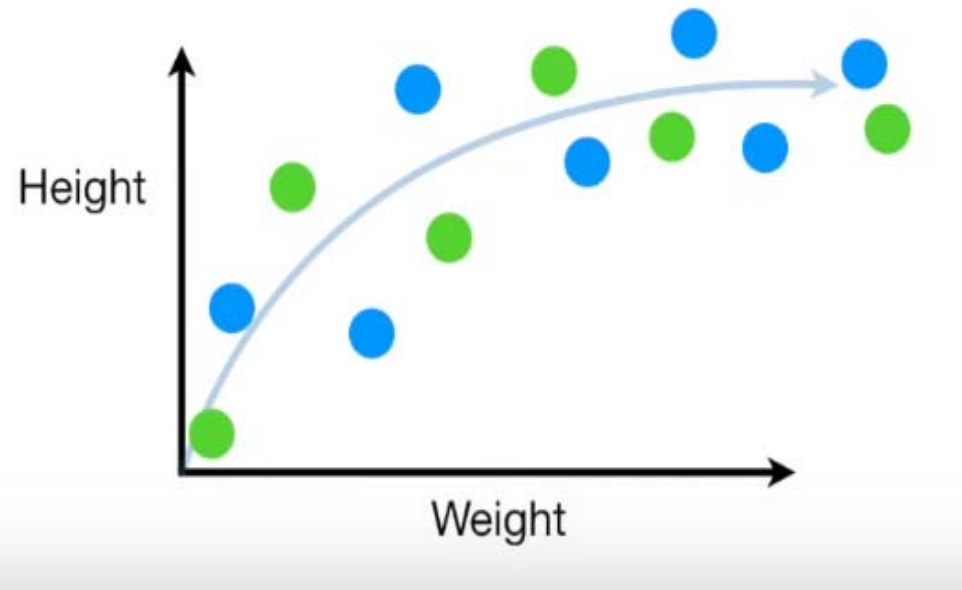


Linear Regression Model

Bias and Variance

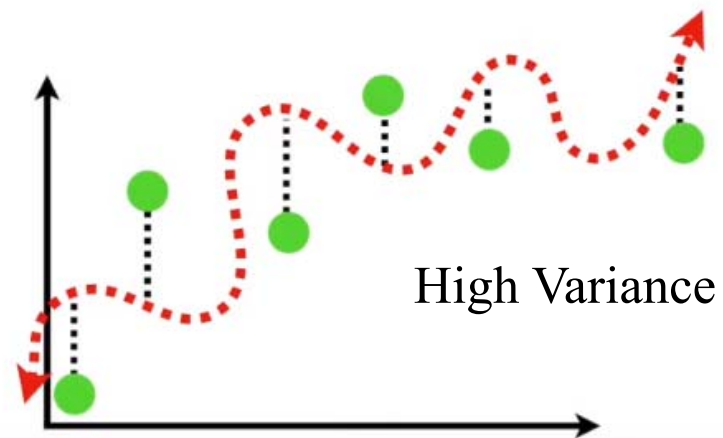
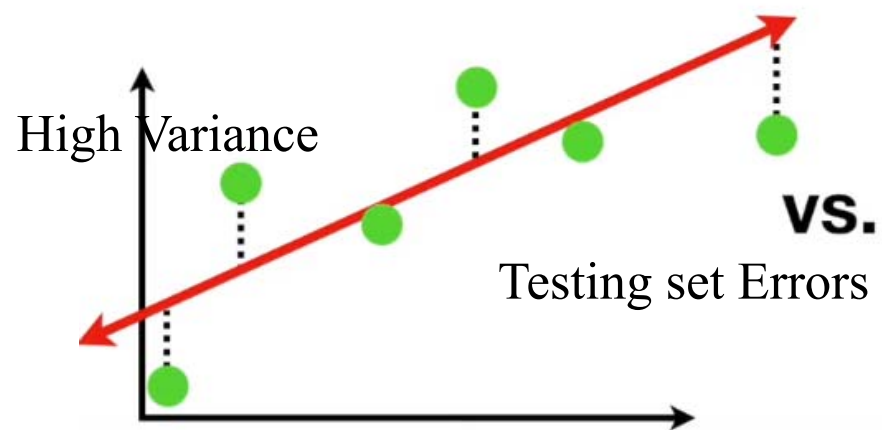
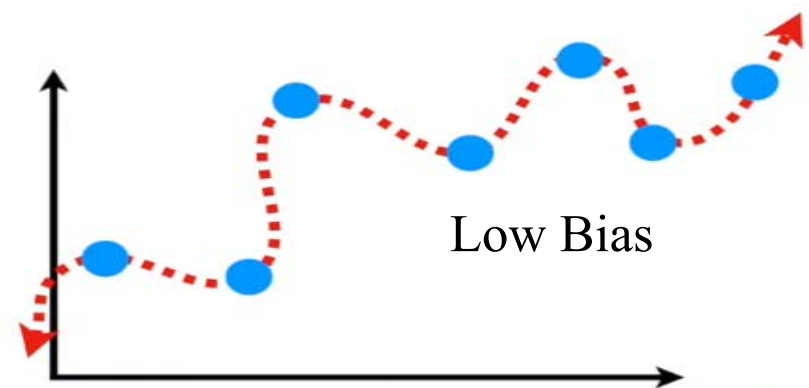
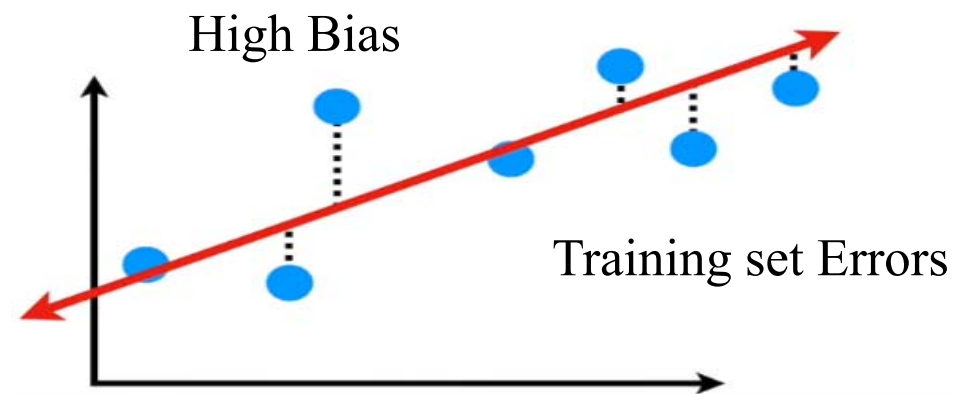


High Degree Polynomial Model



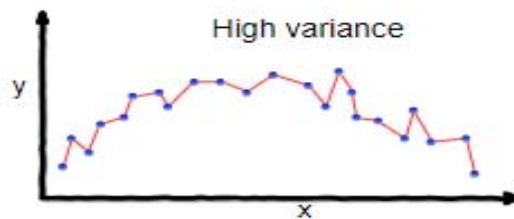
Accurate Model

Bias and Variance

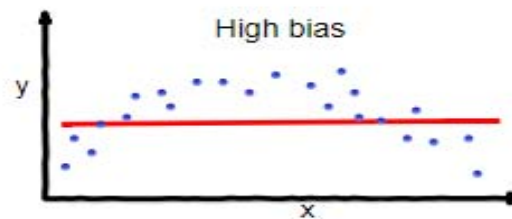


Bias and Variance

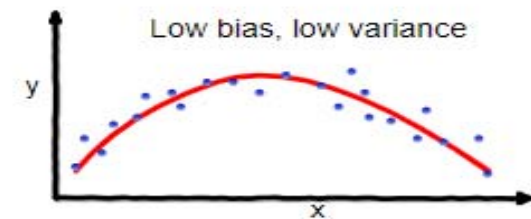
- By measuring the bias and variance on a problem, we can determine how to improve our model
 - If bias is high, we need to allow our model to be more complex
 - If variance is high, we need to reduce the complexity of the model.



overfitting



underfitting



Good balance

How do we estimate performance measures?

- We split the dataset into two parts: *a training and a test dataset*.
- After the machine learning algorithm fit *a model to the training set*, we *evaluated it on the independent test set* that we withheld from the machine learning algorithm during model fitting.
- **Problem:** Finite data are available only and have *to be used both for training and testing*.
 - *More training data gives better generalization*.
 - *More test data* gives *better estimate* for the *classification error probability*.
 - *Never evaluate performance on training data*. The conclusion would be optimistically biased.
- **Error on training data?**
 - Also called re-substitution error.
 - Not a good indicator of the performance on future data.

How do we estimate performance measures?

- *Ideally*, the estimated performance of a model tells *how well it performs on unseen data* - making predictions on *future data is often the main problem* we want to solve in applications of machine learning or the development of new algorithms.
- Running a learning algorithm *over a training dataset with different learning algorithm will result in different models*.
- Since we are typically interested in *selecting the best-performing model* from this set, we need to find a way *to estimate their respective performances in order to rank them against each other*.
- *Methods of Estimating Performance*
 - *Hold out, Cross validation, and Bootstrap.*
- Once evaluation is finished, all the available data can be used to train the final classifier.

Hold out Method

- The holdout method is the *simplest model evaluation technique*; it can be summarized as follows.
 - First, we take a labeled dataset and *split it into two parts: A training and a test set*.
 - Then, we *fit a model to the training data* and *predict the labels of the test set*.
 - The fraction of correct predictions, which can be computed by *comparing the predicted labels to the ground truth labels of the test set*, constitutes our estimate of the model's prediction accuracy.
- We assume that all data points have been drawn from the *same probability distribution (with respect to each class)*. And we randomly choose *2/3 of these samples for the training set and 1/3 of the samples for the test set*.
- Typically, the *splitting of a dataset* into training and test sets is a *simple process of random subsampling*.

Hold out Method

- ***Stratified Holdout:*** Generate holdout using *stratified sampling*.
 - Generates new subsets of instances with an approximately equal proportions of classes.
 - Ensures that the classes are equally represented in the samples
- ***Repeated holdout :***
 - Repeat the ***hold out k times***. In each iteration, a certain proportion of data is randomly selected for training.
 - The ***error rates on different iterations*** are averaged to yield an overall error rate.
 - The accuracy is estimated as the ***average of the accuracies obtained***.
 - ***Improves the reliability*** of the holdout estimate by repeating the process with different subsamples.

Random Subsampling

- When we randomly divide a labeled dataset into training and test sets, we violate the assumption *of statistical independence*.
- The Iris datasets consists of **50 Setosa, 50 Versicolor, and 50 Virginica** flowers; the flower species are distributed uniformly:
 - **33.3% Setosa • 33.3% Versicolor • 33.3% Virginica**
- If a random function *assigns 2/3 of the flowers (100)* to the training set and *1/3 of the flowers (50) to the test set*, it may yield the following
 - **training set** $\rightarrow 38 \times \text{Setosa}, 28 \times \text{Versicolor}, 34 \times \text{Virginica}$
 - **test set** $\rightarrow 12 \times \text{Setosa}, 22 \times \text{Versicolor}, 16 \times \text{Virginica}$
- we just created *two imbalanced datasets* with non-uniform class distributions.
- The **class ratio** that the learning algorithm uses to learn the model is **38% / 28% / 34%**. The test dataset that is used for evaluating the model *is imbalanced as well, and even worse*.

Stratification

- Unless the learning algorithm is *completely insensitive to these perturbations*, this is certainly not ideal.
- The problem becomes *even worse if a dataset has a high class imbalance upfront*, prior to the random subsampling. In the *worst-case scenario*, the test set *may not contain any instance of a minority class* at all.
- Thus, a recommended practice is to divide the dataset in *a stratified fashion*.
- Here, *stratification simply means* that we randomly split a dataset such that each class is *correctly represented in the resulting subsets* (the training and the test set) – in other words, *stratification is an approach to maintain the original class proportion in resulting subsets*.
- It shall be noted that *random subsampling in non-stratified fashion* is usually not a big concern when working with *relatively large and balanced datasets*.

Cross Validation (CV)

- The main idea behind cross-validation is that *each sample in our dataset has the opportunity of being tested*. That means cross-validation ensures *non-overlapping test sets*.
- In *stratified CV*, stratified sampling is used when partitioning the data.
- *In k-fold cross-validation*
 - Widely used approach for estimating test error
 - *Iterate over a dataset set k times.*
 - In each round, we *split the dataset into k parts*: *one part* is used for validation, and the *remaining $k-1$ parts* are merged into a training subset for model evaluation.
 - Compute the cross-validation *performance as the arithmetic mean over the k performance* estimates from the validation sets.
 - The error *estimates are averaged across each of the k folds*.

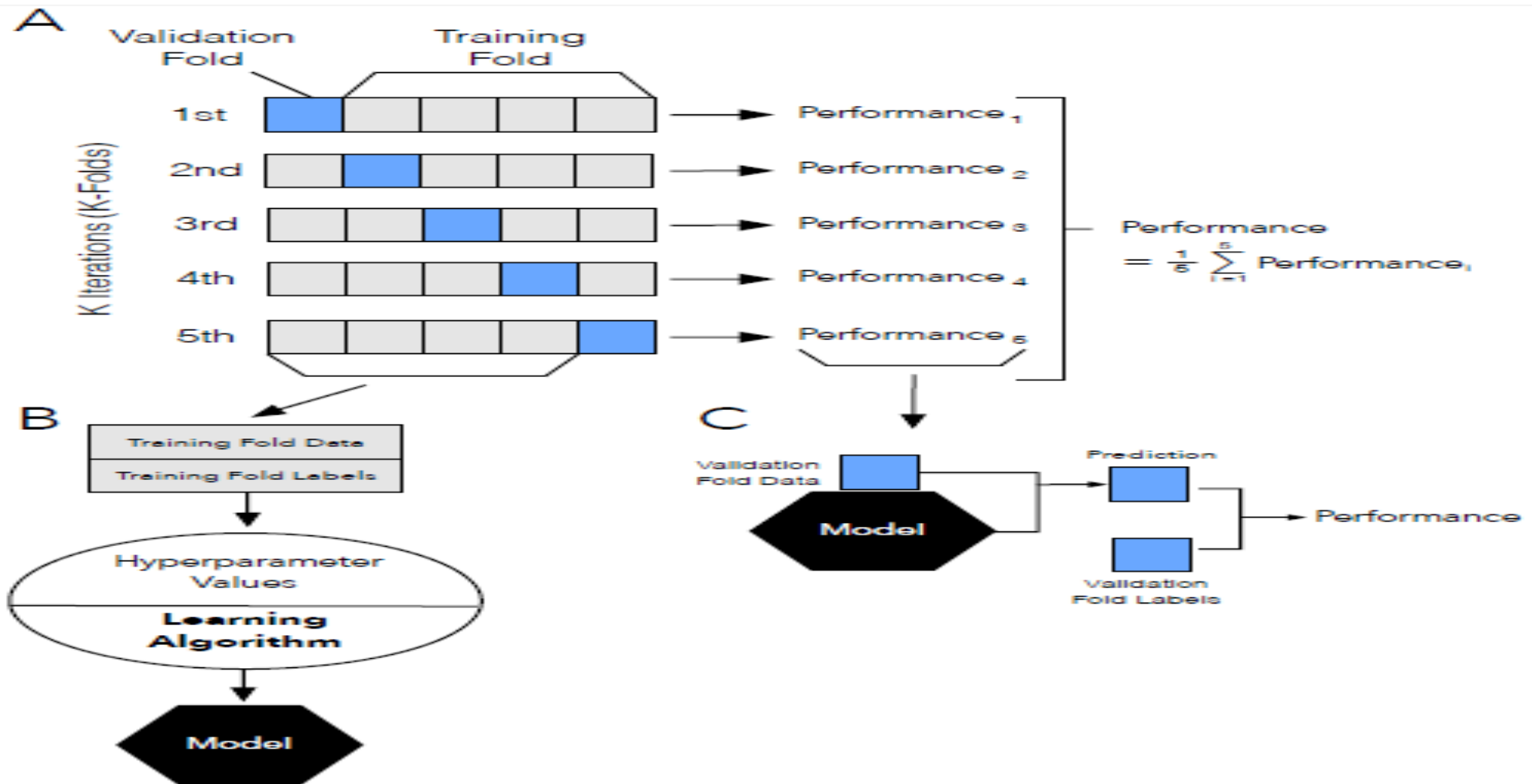
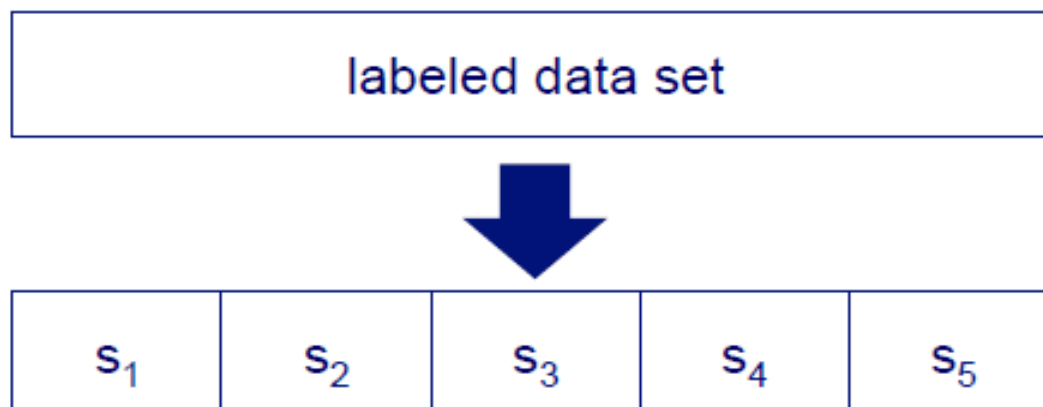


Figure: Illustration of the k-fold cross-validation procedure

Cross Validation (CV)

partition data
into n subsamples



iteratively leave one
subsample out for
the test set, train on
the rest

iteration	train on	test on
1	s_2 s_3 s_4 s_5	s_1
2	s_1 s_3 s_4 s_5	s_2
3	s_1 s_2 s_4 s_5	s_3
4	s_1 s_2 s_3 s_5	s_4
5	s_1 s_2 s_3 s_4	s_5

Cross Validation (CV)

- Suppose we have 100 instances, and we want to estimate accuracy with cross validation.

iteration	train on	test on	correct
1	s ₂ s ₃ s ₄ s ₅	s ₁	11 / 20
2	s ₁ s ₃ s ₄ s ₅	s ₂	17 / 20
3	s ₁ s ₂ s ₄ s ₅	s ₃	16 / 20
4	s ₁ s ₂ s ₃ s ₅	s ₄	13 / 20
5	s ₁ s ₂ s ₃ s ₄	s ₅	16 / 20

$$\text{accuracy} = 73/100 = 73\%$$

Leave-One-Out Cross-Validation (LOOCV)

- A special case of K -fold cross validation *with $K = n$* , where n is the total number of samples in the training multiset.
- *n experiments are performed using $n - 1$ samples for training and the remaining sample for testing.*
- *In each iteration* during LOOCV, we fit a *model to $n-1$ samples of the dataset and evaluate it on the single, remaining data point.*
- Although this *process is computationally expensive*, given that we have n iterations, it can be useful for small datasets, cases where withholding data from the training set would be too wasteful.
- A *disadvantage* of Leave-One-Out-CV is that *stratification is not possible*: It guarantees a non-stratified sample because there is only one instance in the test set!

Leave-One-Out Cross-Validation (LOOCV)

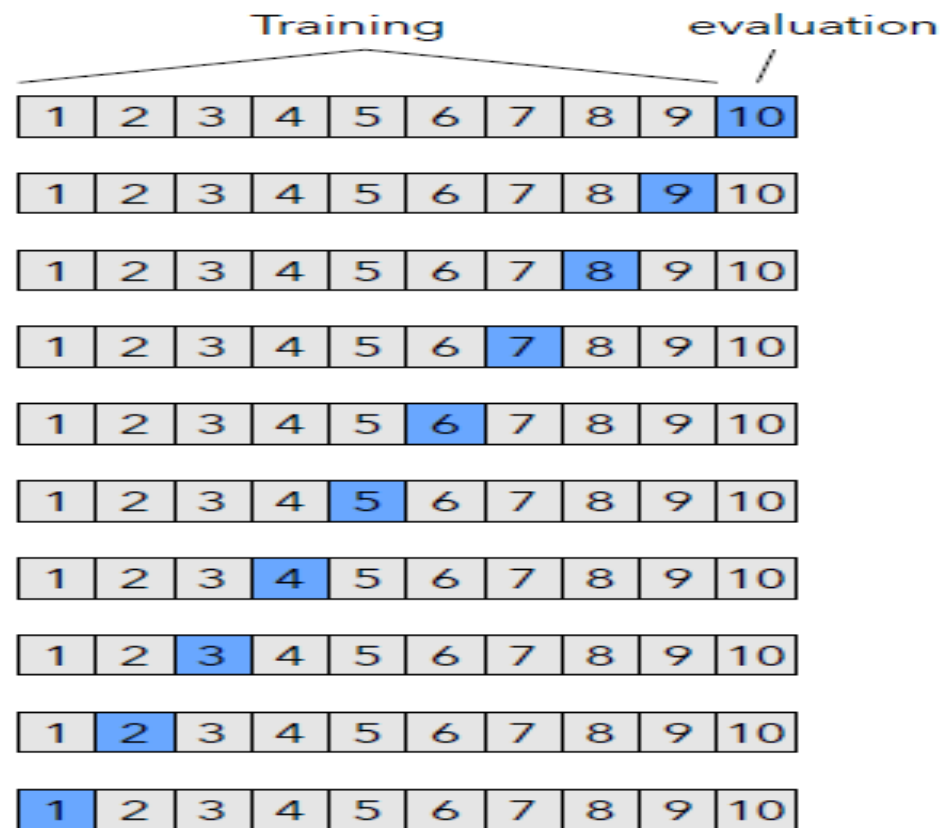


Figure 15: Illustration of leave-one-out cross-validation.

Bias-Variance Trade-off for k-fold Cross-Validation

- *Which is better, LOOCV or K-fold CV?*
 - LOOCV is more computationally intensive than K -fold CV
 - From the perspective of bias reduction, LOOCV is preferred to K -fold CV (when $K < n$)
 - However, LOOCV has higher variance than K -fold CV (when $K < n$)
- *The general trends when increasing the number of folds or k*
 - *The bias of the performance estimator decreases (more accurate)*
 - *The variance of the performance estimators increases (more variability)*
 - The computational cost increases (more iterations, larger training sets during fitting)
 - **Exception:** decreasing the value of **k in k -fold cross-validation** to small values (for example, 2 or 3) also **increases the variance on small datasets** due to random sampling effects.

How many folds are needed?

- In practice, the *choice of the number of folds depends on the size of the dataset*
 - *For large datasets*, even *3-Fold* Cross Validation will *be quite accurate*.
 - For *very sparse datasets*, we may *have to use leave-one-out* in order to train on as many examples as possible.
- We tend to use *K-fold CV with $K = 5$ or $K = 10$* , as these values have been shown empirically to yield test error rate estimates that suffer neither from excessively high bias nor from very high variance

CV-based Model Selection

- **Step 1.** Split the dataset into two parts, *a training and an independent test set*;
- **Step 2.** In this second step, we can now *experiment with various ML algorithms*; we could use Bayesian optimization, randomized search, or grid search, for example. *For each ML algorithms, we apply the k-fold cross-validation method* on the training set, resulting in multiple models and performance estimates.
- **Step 3.** Taking the hyperparameter settings that *produced the best results in the k-fold cross validation procedure*, we can then *use the complete training set for model fitting* with these settings.
- **Step 4.** Now, we *use the independent test set* that we withheld earlier (Step 1); we use this test set to evaluate the model that we obtained from Step 3.
- **Step 5.** Finally, after we completed the evaluation stage, we can *optionally fit a model to all data* (training and test datasets combined), which *could be the model for (the so-called) deployment*.

CV-based Model Selection

Example: Choosing “k” for a k-nearest-neighbor regression.

Compute 10-fold-CV-ERR error for six different model classes:

Algorithm	TRAINERR	10-fold-CV-ERR	Choice
K=1			
K=2			
K=3			
K=4			✓
K=5			
K=6			

Whichever model class gave best CV score: train it with all the data, and that's the predictive model you'll use.

CV-based Model Selection

Example: Choosing “k” for a k-nearest-neighbor regression.

Compute LOOCV error for six different model classes

Algorithm	TRAINERR	LOOCV-ERR	Choice
$K=1$			
$K=2$			
$K=3$			
$K=4$			✓
$K=5$			
$K=6$			

Whichever model class gave best CV score: train it with all the data, and that's the predictive model you'll use.

Bootstrap

- Bootstrap is a *resampling technique*. It *generates multiple datasets by sampling from a single, original dataset*. Each of the “new” datasets can be used to estimate a quantity of interest. Since there are multiple datasets and therefore multiple estimates, can also calculate variance or a confidence interval for the estimate.
- Bootstrap is closely related to cross-validation. It was *inspired by another resampling technique called the jackknife*, which is essentially leave-one-out cross-validation.
- Bootstrap *resamples the data with replacement*. Given a dataset containing N data points, bootstrap picks a data point uniformly at random, adds it to the bootstrapped set, *puts that data point back into the dataset*, and repeats.
- Bootstrapping *replicates the process of sample generation from an underlying population* by drawing samples with replacement from the original data set, *of the same size as the original data set*.

Bootstrap

Original Dataset

x_1	x_2	x_3	x_4	x_5	x_6	x_7	x_8	x_9	x_{10}
-------	-------	-------	-------	-------	-------	-------	-------	-------	----------

Bootstrap 1

x_8	x_6	x_2	x_9	x_5	x_8	x_1	x_4	x_8	x_2
-------	-------	-------	-------	-------	-------	-------	-------	-------	-------

x_3	x_7	x_{10}
-------	-------	----------

Bootstrap 2

x_{10}	x_1	x_3	x_5	x_1	x_7	x_4	x_2	x_1	x_8
----------	-------	-------	-------	-------	-------	-------	-------	-------	-------

x_6	x_9
-------	-------

Bootstrap 3

x_6	x_5	x_4	x_1	x_2	x_4	x_2	x_6	x_9	x_2
-------	-------	-------	-------	-------	-------	-------	-------	-------	-------

x_3	x_7	x_8	x_{10}
-------	-------	-------	----------

Training Sets

Test Sets

Bootstrap

- *Models may be developed in bootstrap samples and tested in the original sample.*
- Regular bootstrap: the model as estimated in the bootstrap sample was evaluated in the bootstrap sample and in the original sample. *The performance in the bootstrap sample represents estimation of the apparent performance, and the performance in the original sample represents test performance.* The difference between these performances is an estimate of the optimism in the apparent performance.
- This difference is averaged to obtain a stable estimate of the optimism. internally validated performance .
- *Optimism = average (bootstrap performance – test performance).*
- *Estimated performance = apparent performance – optimism.*

Thank You

???

■ *References:*

- 1) R.O.Duda, P.E.Hart and D.G.Stork, Pattern Classification, John Wiley, 2001
1. S.Theodoridis and K.Koutroumbas, Pattern Recognition, Academic Press, 2009
2. C.M.Bishop, Pattern Recognition and Machine Learning, Springer, 2006
3. Kevin P. Murphy ,Machine Learning A Probabilistic Perspective